

Python 常用数据结构

0. 对比

类型	字面量	可变	是否有序	元素可重复	作为字典键
list 列表	<code>[]</code>	可变	有序	可	否
tuple 元组	<code>()</code>	不可变	有序	可	可（若内部全可哈希）
dict 字典	<code>{k: v}</code>	可变	有序（Py3.7+保持插入顺序）	键不可重复（后者覆盖前者）	—
set 集合	<code>{ } / set()</code>	可变	无序	否（自动去重）	—

空集合请用 `set()`；`{ }` 是空字典。

1) list 列表

创建与访问

```
1 nums = [1, 2, 3]
2 xs = list("abc")      # ['a', 'b', 'c']
3 xs[0], xs[-1]         # 索引：-1 为最后一个
4 xs[1:3]               # 切片（不含右端）
```

常用操作

```
1 xs.append(4)           # 尾部追加
2 xs.extend([5, 6])      # 连接
3 xs.insert(1, 99)       # 指定位置插入
4 xs.pop()               # 弹尾部并返回
5 xs.pop(1)              # 删除索引1
6 xs.remove(99)           # 按值删首个匹配
7 xs.clear()
```

排序与反转

```

1 xs = [3,1,2]
2 xs.sort()           # 就地排序, 返回 None
3 xs.sort(key=abs, reverse=True)
4 ys = sorted(xs)     # 生成新列表
5 xs.reverse()        # 就地反转

```

列表推导式

```

1 squares = [i*i for i in range(5) if i%2==0]

```

切片与切片赋值

```

1 xs = [0,1,2,3]
2 xs[1:3] = [10, 11, 12] # 可改变长度

```

复制与乘法

```

1 a = [[0]*3]*2           # 错误: 两行指向同一子列表
2 b = [[0 for _ in range(3)] for _ in range(2)] # 正确

```

* 复制的是引用。

```

1 a = [[0]*3]*2
2 a[0] is a[1]           # True, 同一对象
3 a[0][0] = 1
4 print(a)               # [[1, 0, 0], [1, 0, 0]] 两行一起变
5
6 row = [0]*3: 创建一个新列表, 元素是对同一个 int 0 的引用。int 不可变

```

2) tuple 元组

基本

```

1 t = (1, 2, 3)
2 single = (1,)           # 单元素必须带逗号
3 empty = ()

```

- 不可变, 有序。可作为字典键 (内部元素必须可哈希)。

打包与解包

```

1 a, b = 1, 2             # 解包赋值
2 x = (1, 2)
3 a, b = x
4 # 交换
5 a, b = b, a

```

3) dict 字典 (map)

创建

```
1 d = {"a":1, "b":2}
2 d = dict([("a",1), ("b",2)])
3 d = dict(a=1, b=2)
```

- 键必须可哈希：常见为 `str`、`int`、`tuple`(不可变元素)。

访问与更新

```
1 d["a"]          # KeyError 若不存在
2 v = d.get("a", 0) # 不存在返回默认值
3
4 d["c"] = 3       # 新增/更新
5
6 d.update({"a":10, "d":4})
7 d |= {"e":5}     # 3.9+ 合并
```

删除

```
1 d.pop("a", None) # 删除并返回键 "a" 对应的值；若不存在则返回 None，不报错
2 key, val = d.popitem() # 从字典中删除并返回一对键值，默认按后进先出（LIFO），即删除最后插入的那一对。字典为空时抛 KeyError
```

遍历

```
1 for k in d: ...           # 键
2 for k, v in d.items(): ... # 键值
3 for v in d.values(): ...
4 "a" in d                  # 仅查键
```

有序性

- Python3.7+ 规定保持**插入顺序**（语言保证）。

4) set 集合（去重）

创建与基本操作

```
1 s = {1,2,3}
2 s = set([1,2,2,3]) # {1,2,3}
3
4 s.add(4)
5 s.update([3,5])
6 s.discard(10)      # 不存在不报错
7 # s.remove(10)     # 不存在会报错
8 s.pop()             # 随机弹出一个元素
```

集合运算

```
1 a, b = {1,2,3}, {3,4}
2 a | b      # 并集 {1,2,3,4}
3 a & b      # 交集 {3}
4 a - b      # 差集 {1,2}
5 a <= b     # 子集
```

```
1 s = "banana"
2 #生成字符频次表
3 cnt = {ch: s.count(ch) for ch in set(s)}
```

易错点

- `{}` 是字典，空集合用 `set()`；元素必须可哈希。

5) 拷贝与共享引用

```
1 import copy
2
3 # 浅拷贝：顶层复制，内部引用共享
4 l1 = [[1],[2]]
5 l2 = l1.copy()      # 或 l1[:] / list(l1)
6 l2[0].append(99)
7 # l1[0] 也会受影响
8
9 # 深拷贝：递归复制
10 l3 = copy.deepcopy(l1)
```

函数默认参数不要用可变对象：`def f(x, cache=None): cache = cache or {} ...`

6) 常见模式

6.1 去重并保持原顺序

```
1 seen = set()
2 res = []
3 for x in [1,2,2,3,1]:
4     if x not in seen:
5         seen.add(x)
6         res.append(x)
7 # res = [1,2,3]
```

6.2 计数

```
1 from collections import Counter
2 Counter("banana")      # Counter({'a':3,'n':2,'b':1})
```

if语句

1. 基本语法

```
1 if 条件:
2     代码块
```

- 条件为布尔结果 (True/False) 。
- 末尾必须有冒号 `:`。
- 代码块使用缩进表明从属关系。

示例:

```
1 age = 20
2 if age >= 18:
3     print("adult")
```

2. if / elif / else 结构

```
1 if 条件1:
2     ...
3 elif 条件2:
4     ...
5 else:
6     ...
```

- 先匹配到就执行，不再继续匹配。
- `elif` 可有多； `else` 可省略。

示例:

```
1 score = 85
2 if score >= 90:
3     grade = "A"
4 elif score >= 80:
5     grade = "B"
6 elif score >= 70:
7     grade = "C"
8 elif score >= 60:
9     grade = "D"
10 else:
11     grade = "F"
12 print(grade)
```

3. 缩进与代码块

- 推荐每级4个空格。
- 不要混用 Tab 与空格。
- 缩进不一致会报 `IndentationError`。

示例：

```
1 | n = 3
2 | if n > 0:
3 |     print("positive")    # 同一层级保持一致缩进
4 |     print("done")
```

4. 布尔与“真值”

- 直接布尔：`True`, `False`。
- 能转成布尔的对象：非零数字、非空容器为真；0、空串、空列表、空字典、`None` 为假。

示例：

```
1 | if "hello":
2 |     print("非空字符串为真")
3 | if 0:
4 |     print("不会执行")
```

常见“假”值：`False`, `None`, `0`, `0.0`, `0j`, `""`, `[]`, `{}`, `set()`, `range(0)`。

5. 比较与逻辑运算符

- 比较：`==` `!=` `<` `<=` `>` `>=`
- 逻辑：`and`、`or`、`not`
- 链式比较：`18 <= age < 65`

示例：

```
1 | x = 10
2 | if 1 < x <= 10 and not (x % 2):
3 |     print("x在(1,10]且是偶数")
```

6. 多条件常用写法

区间判断（链式比较）：

```
1 | if 0 <= score <= 100:
2 |     print("有效分数")
```

成员判断：

```
1 | op = input("选择运算(+ - * /): ")
2 | if op in ["+", "-", "*", "/"]:
3 |     print("合法运算符")
```

任意/全部条件：

```
1 u, v, w = True, False, True
2 if any([u, v, w]):
3     print("至少一个为真")
4 if all([u, w]):
5     print("全部为真")
```

7. 嵌套与组合

```
1 age = 25
2 is_member = True
3 if age >= 18:
4     if is_member:
5         price = 50
6     else:
7         price = 80
8 else:
9     price = 30
10 print(price)
```

建议：能用 `elif` 或计算式合并时，就避免深层嵌套。

8. 条件表达式（三元运算）

```
1 # 语法: A if 条件 else B
2 age = 20
3 status = "adult" if age >= 18 else "minor"
```

适合简单赋值场景，避免写成多行。

9. 输入与类型转换

`input()` 返回字符串。数值判断需先转型。

```
1 raw = input("输入年龄: ")
2 if raw.isdigit(): # 字符串非空且全部是数字字符
3     age = int(raw)
4     if age >= 18:
5         print("adult")
6 else:
7     print("非法输入")
```

10. 常见错误与规避

1. `=` 与 `==` 混淆：

```
1 # 错误: if x = 1:
2 # 正确:
3 if x == 1:
4     ...
```

- 漏写冒号 `:`。
- 缩进不一致或混用 Tab/空格。
- 用 `is` 比较数值或字符串：`is` 比身份，`==` 比值。

```
1 # 不要这样: if x is 1:
2 # 应该:
3 if x == 1:
4     ...
```

1. 把 `and/or` 写成按位运算 `& / |`: 布尔逻辑应使用 `and/or`。
2. 浮点数直接相等比较:

```
1 x = 0.1 + 0.2
2 if abs(x - 0.3) < 1e-9:
3     print("近似相等")
```

1. 类型未统一比较: `"3" == 3` 为 `False`, 先转换类型。

11. 典型模式

守卫式返回 (在函数中早退出) :

```
1 def safe_div(a, b):
2     if b == 0:
3         return None
4     return a / b
```

14. 速查清单

- 结构: `if / elif / else`, 末尾冒号。
- 缩进: 每级4空格, 禁用 Tab。
- 常用: 链式比较、`in`、`any/all`、三元表达式。
- 避坑: `=` vs `==`; 不要用 `is` 比值; 避免浮点直接相等; 逻辑用 `and/or`。